[메뉴얼]Carla맵 생성

상태	시작 전
선택	메뉴얼

1. generation

   https://carla.readthedocs.io/en/latest/tuto_M_generate_map/

2. importation

   https://carla.readthedocs.io/en/latest/tuto_M_add_map_source/

   https://carla.readthedocs.io/en/latest/tuto_M_add_map_alternative/#manual-import

3. customization
4. generate pedestrian navigation

.xodr파일만 가지고 임시 맵을 만드는거(standalone 개잡기술)

: 3D Mesh 구조물이 없는 도로만 있는 임시맵을 일시적으로 띄운다

https://carla.readthedocs.io/en/latest/adv_opendrive/

그 뭐냐 xodr파일 만드는거

1. pythonapi함수중에서 world = client.generate_opendrive_world(xodr_text, params) 이게 있는데 이건 xodr파일만 받고 자동으로 도로만 만들어준다

   ▼ 예시코드

```
#!/usr/bin/env python3
import argparse
from pathlib import Path
import carla
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--host", default="localhost")
    ap.add_argument("--port", type=int, default=2000)
    ap.add_argument("--xodr", required=True, help="Path to .xod
r file")
    # OSM에서 만든 xodr이면 wall_height=0을 추천(벽 겹침 문제) :con
tentReference[oaicite:2]{index=2}    ap.add_argument("--wall-he
ight", type=float, default=1.0)
    ap.add_argument("--vertex-distance", type=float, default=2.
0)
```

```
    ap.add_argument("--max-road-length", type=float, default=5
0.0)
    ap.add_argument("--additional-width", type=float, default=
0.6)
    ap.add_argument("--no-smooth-junctions", action="store_tru
e")
    ap.add_argument("--invisible-mesh", action="store_true")
    args = ap.parse_args()

    client = carla.Client(args.host, args.port)    # 월드 생성이
오래 걸릴 수 있어 timeout 넉넉히 :contentReference[oaicite:3]{inde
x=3}    client.set_timeout(120.0)

    xodr_text = Path(args.xodr).read_text(encoding="utf-8")

    params = carla.OpendriveGenerationParameters(
        vertex_distance=float(args.vertex_distance),
        max_road_length=float(args.max_road_length),
        wall_height=float(args.wall_height),
        additional_width=float(args.additional_width),
        smooth_junctions=not args.no_smooth_junctions,
        enable_mesh_visibility=not args.invisible_mesh,
        )

    world = client.generate_opendrive_world(xodr_text, params)
# 성공하면 서버에 새 월드가 로드된 상태로 남음    print("OK: OpenDRIV
E world generated.")    print("Map name:", world.get_map().nam
e)
if __name__ == "__main__":
    main()
```

2. xodr파일은 xml파일같은거 그냥 도로정보만 있음
3. gui로 xodr파일 만들어주는 툴 → lanemaker, Truevisiondesigner

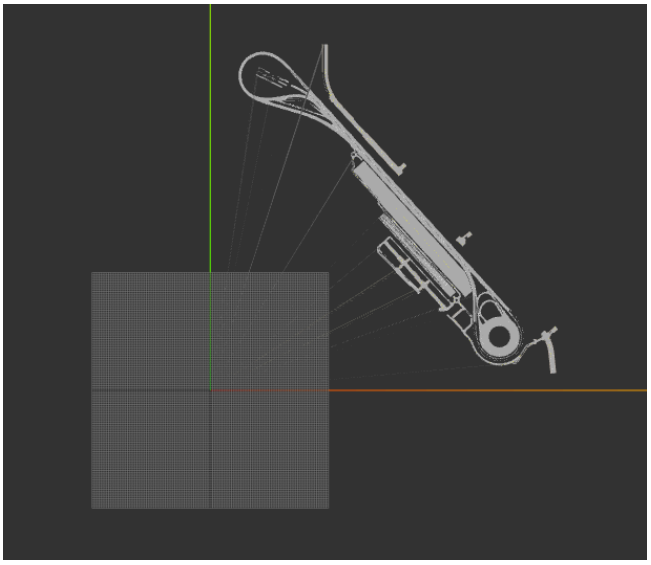
---

cd ~/Downloads./TruevisionDesigner-2024.9.24.AppImage

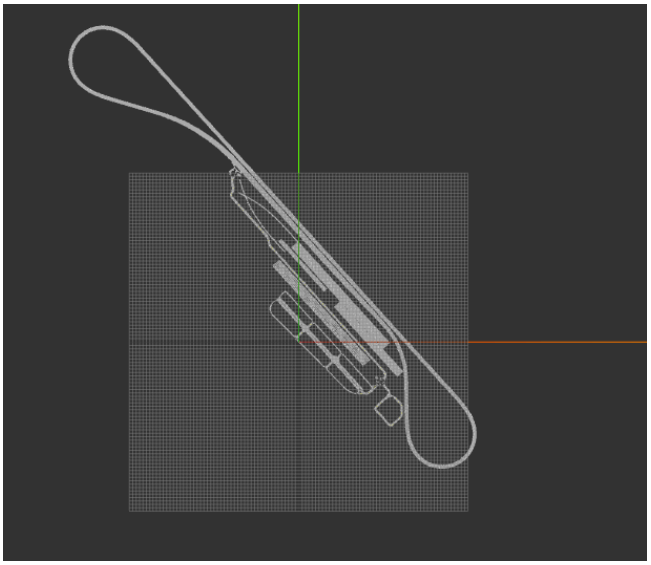
실행명령어

- kiapi.xodr파일

  https://pg.kiapi.or.kr/intro/testRoadView.do?road=02



- [230919]KIAPI.xodr



xodr파일로 fbx파일 얻기

진짜 “libOpenDRIVE 다운로드부터 → xodr2obj 빌드 → kiapi.xodr를 OBJ로 → Blender로 FBX”까지 과정

▼ 코드

```
#!/usr/bin/env python3
"""
xodr_to_fbx.py
Convert OpenDRIVE .xodr -> .fbx using:
```

```
  1) xodr2obj (libOpenDRIVE-based converter you built)
  2) Blender (background) to export FBX

Usage example:
  python3 xodr_to_fbx.py \
    --xodr /mnt/hdd/carla/PythonAPI/examples/ros2/world/kiapi.xodr
\
    --fbx  /mnt/hdd/out_kiapi/kiapi.fbx \
    --xodr2obj /mnt/hdd/xodr_tools/xodr2obj/build/xodr2obj \
    --blender  /mnt/hdd/blender-5.0.1-linux-x64/blender \
    --eps 0.2
"""

from __future__ import annotations
import argparse
import os
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path
import textwrap

BLENDER_HELPER = r"""
import bpy
import sys
from pathlib import Path

argv = sys.argv
argv = argv[argv.index("--") + 1:] if "--" in argv else []
if len(argv) < 4:
    raise SystemExit("Usage: blender --background --python helper.
py -- input.obj output.fbx axis_forward axis_up")

in_obj = Path(argv[0]).resolve()
out_fbx = Path(argv[1]).resolve()
axis_forward = argv[2]
axis_up = argv[3]

bpy.ops.wm.read_factory_settings(use_empty=True)

# Import OBJ (Blender 4/5 uses wm.obj_import; fallback for older)
imported = False
try:
```

```
    bpy.ops.wm.obj_import(filepath=str(in_obj))
    imported = True
except Exception:
    pass
if not imported:
    bpy.ops.import_scene.obj(filepath=str(in_obj))

# Select mesh objects
mesh_objs = [o for o in bpy.context.scene.objects if o.type == "ME
SH"]
if not mesh_objs:
    raise SystemExit("No MESH objects imported from OBJ. (OBJ migh
t be empty)")

for o in mesh_objs:
    o.select_set(True)
bpy.context.view_layer.objects.active = mesh_objs[0]

# Apply rotation/scale so exported FBX is clean
bpy.ops.object.transform_apply(location=False, rotation=True, scal
e=True)

# Export FBX
bpy.ops.export_scene.fbx(
    filepath=str(out_fbx),
    use_selection=False,
    apply_unit_scale=True,
    bake_space_transform=True,
    axis_forward=axis_forward,
    axis_up=axis_up,
)

print(f"Exported FBX: {out_fbx}")
"""

def run(cmd: list[str], *, cwd: Path | None = None) -> None:
    print("\n$ " + " ".join([shlex_quote(c) for c in cmd]))
    subprocess.run(cmd, cwd=str(cwd) if cwd else None, check=True)

def shlex_quote(s: str) -> str:
    # minimal safe quoting for printing
    if not s:
        return "''"
    if any(ch in s for ch in " \t\n\"'\\$`!()[]{};|&<>?*"):
```

```
        return "'" + s.replace("'", "'\"'\"'") + "'"
    return s

def main() -> int:
    ap = argparse.ArgumentParser(
        formatter_class=argparse.RawTextHelpFormatter,
        description="Convert .xodr to .fbx using xodr2obj + Blende
r."
    )
    ap.add_argument("--xodr", required=True, help="Input .xodr pat
h")
    ap.add_argument("--fbx", required=True, help="Output .fbx pat
h")
    ap.add_argument("--eps", type=float, default=0.2, help="Mesh s
ampling step for xodr2obj (default: 0.2)")
    ap.add_argument("--xodr2obj", default="/mnt/hdd/xodr_tools/xod
r2obj/build/xodr2obj",
                    help="Path to xodr2obj binary")
    ap.add_argument("--blender", default="/mnt/hdd/blender-5.0.1-l
inux-x64/blender",
                    help="Path to Blender executable (or blender-s
oftwaregl)")
    ap.add_argument("--axis-forward", default="-Y", help="FBX expo
rt axis_forward (default: -Y)")
    ap.add_argument("--axis-up", default="Z", help="FBX export axi
s_up (default: Z)")
    ap.add_argument("--keep-obj", action="store_true", help="Keep
intermediate OBJ next to output FBX")
    args = ap.parse_args()

    xodr = Path(args.xodr).expanduser().resolve()
    fbx = Path(args.fbx).expanduser().resolve()
    xodr2obj = Path(args.xodr2obj).expanduser().resolve()
    blender = Path(args.blender).expanduser().resolve()

    if not xodr.exists():
        print(f"[ERROR] XODR not found: {xodr}", file=sys.stderr)
        return 2
    if not xodr2obj.exists():
        print(f"[ERROR] xodr2obj not found: {xodr2obj}", file=sys.
stderr)
        print("  -> 먼저 xodr2obj를 빌드한 뒤, --xodr2obj로 경로를 넘
겨주세요.", file=sys.stderr)
        return 3
```

```
    if not blender.exists():
        print(f"[ERROR] Blender not found: {blender}", file=sys.st
derr)
        print("  -> --blender로 blender(또는 blender-softwaregl) 경
로를 넘겨주세요.", file=sys.stderr)
        return 4

    fbx.parent.mkdir(parents=True, exist_ok=True)

    with tempfile.TemporaryDirectory(prefix="xodr_to_fbx_") as td:
        tmp = Path(td)
        helper_py = tmp / "blender_obj_to_fbx.py"
        helper_py.write_text(BLENDER_HELPER, encoding="utf-8")

        if args.keep_obj:
            obj = fbx.with_suffix(".obj")
        else:
            obj = tmp / (fbx.stem + ".obj")

        # 1) XODR -> OBJ
        cmd1 = [str(xodr2obj), str(xodr), str(obj), str(args.eps)]
        try:
            run(cmd1)
        except subprocess.CalledProcessError:
            print("[ERROR] xodr2obj failed. (Check XODR compatibil
ity / eps)", file=sys.stderr)
            return 10

        if not obj.exists() or obj.stat().st_size < 100:
            print(f"[ERROR] OBJ not generated or too small: {ob
j}", file=sys.stderr)
            return 11

        # 2) OBJ -> FBX (Blender)
        cmd2 = [
            str(blender),
            "--background",
            "--python", str(helper_py),
            "--",
            str(obj),
            str(fbx),
            str(args.axis_forward),
            str(args.axis_up),
        ]
```

```
        try:
            run(cmd2)
        except subprocess.CalledProcessError:
            print("[ERROR] Blender export failed.", file=sys.stder
r)
            return 20

    if not fbx.exists() or fbx.stat().st_size < 1000:
        print(f"[ERROR] FBX not generated or too small: {fbx}", fi
le=sys.stderr)
        return 21

    print(f"\n[DONE] FBX created: {fbx}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

▼ 실행명령

```
python3 /mnt/hdd/out_kiapi/xodr2fbx.py --xodr /mnt/hdd/out_kiapi/p
arkinglot.xodr --fbx  /mnt/hdd/out_kiapi/parkinglot.fbx --xodr2obj
/mnt/hdd/xodr_tools/xodr2obj/build/xodr2obj --blender /mnt/hdd/ble
nder-5.0.1-linux-x64/blender --eps 0.2
```

맵 import 방법

1. **${CARLA_ROOT}/Import 안에 파일 넣어준다**

```
cp ./${my_map}.fbx  Import/${my_map}/${my_map}/${my_map}.fbx
cp ./${my_map}.xodr Import/${my_map}/${my_map}/${my_map}.xodr

cat > Import/${my_map}/${my_map}.json << 'EOF'
{
  "maps": [
    {
      "name": "${my_map}",
      "source": "./${my_map}/${my_map}.fbx",
      "use_carla_materials": true,
      "xodr": "./${my_map}/${my_map}.xodr"
    }
  ],
  "props": []
```

```
}
EOF
```

2. Import폴더 내에 import할 패키지 폴더 말고는 싹다 삭제. 부정탈 위험 상당히 높음
3. make import로 빌드실행
4. 빌드 잘 되었는지 확인

   ▼ 빠른확인

   ```
   cd /path/to/carla
   find Unreal/CarlaUE4/Content -name "*.umap" | grep -i ${my_map}
   ```

Carla 맵 에디터에서 수정하기

- 땅생성방법
  1. plane으로 적절한 사이즈를 배치한다. z값은 -0.5정도
  2. material은 아무거나 선택한다. ground, terrain, dirt 등등. 이후 plane을 작은 타일들로 나누어 준다
     a. 머티리얼 그래프 빈 곳 **우클릭**
     b. `TextureCoordinate` 검색해서 추가
     c. `TextureCoordinate` 노드 클릭 → 왼쪽 Details에서
        - **UTiling / VTiling** 값을 20~50 정도로 올려보기
     d. `TextureCoordinate` 출력(오른쪽 핀)을

        각 Texture Sample의 `UVs` 핀에 연결

        (BaseColor 텍스처, Normal 텍스처 등 전부)

     > 핵심: **UVs 핀에 뭔가 연결돼야** 타일링이 적용됩니다.

- 맵폴더 위치

  a01004@a01004-MS-7C82:/mnt/hdd/carla/Unreal/CarlaUE4/Content/parkinglot$
  맵 파일은 내 usb안에 잇음

Carla패키지에 맵 등록하기

0. 관련 링크

   https://carla.readthedocs.io/en/latest/tuto_content_authoring_maps/#exporting-a-map
1. 맵 파일들(.uamp, .xodr)을 CARLA 표준 경로로 넣어주기

   /path/to/carla/Unreal/CarlaUE4/Content/Carla/Maps/`${my_map}`/`${my_map}`.umap

/path/to/carla/Unreal/CarlaUE4/Content/Carla/Maps/`${my_map}`/OpenDrive/`${my_map}`.xodr

- 자기 맵 폴더를 만들어준다

2. 패키지 설정 파일 하나 만들기

   (/path/to/carla/Unreal/CarlaUE4/Content/Carla/Config) 경로에 json파일으 넣어준다

   파일: `${my_map}.Package.json`

   ▼ ${my_map}.Package.json

```
{
  "props": [],
  "maps": [
    {
      "name": "${my_map}",
      "path": "/Game/Carla/Maps/${my_map}",
      "use_carla_materials": true
    }
  ]
}
```

3. MapsToCook에 한 줄 추가파일: DefaultGame.ini[/Script/UnrealEd.ProjectPackagingSettings] 섹션에 추가:

   `+MapsToCook=(FilePath="/Game/Carla/Maps/${my_map}/${my_map}")`

4. 패키징 실행

   `make package`

   혹은

   `cd /path/to/carla`
   `./Util/BuildTools/Package.sh --packages ${my_map}`

   또는 전체 패키지면 make package.

5. 맵을 패키징후 서버에 임포팅한다

   `cp /path/to/carla/Dist/${my_map}_9c620141f-dirty.tar.gz /mnt/hdd/carla/Dist/Import/`

   `cd /path/to/carla/Dist`
   `./ImportAssets.sh`

   경우에 따라서 Dist안에 ImportAssets.sh없는 경우가 있음. 그러면 하위 디렉토리에 패키지 폴더가 있을테니, 거기로 들어가서 위 스크립트를 찾아서 실행한다

5. 서버 재시작 후 확인

   ▼ 확인코드

```
#!/usr/bin/env python3
import argparse
import sys

try:
    import carla
except ImportError:
    print("carla 모듈을 찾을 수 없습니다. CARLA Python API 환경을
먼저 설정하세요.")
    sys.exit(1)


def main():
    parser = argparse.ArgumentParser(description="CARLA 사용 가
능 맵 목록 출력")
    parser.add_argument("--host", default="localhost", help="CA
RLA 서버 주소")
    parser.add_argument("--port", type=int, default=2000, help
="CARLA RPC 포트")
    parser.add_argument("--timeout", type=float, default=5.0, h
elp="연결 타임아웃(초)")
    args = parser.parse_args()

    client = carla.Client(args.host, args.port)
    client.set_timeout(args.timeout)

    world = client.get_world()
    current_map = world.get_map().name
    available_maps = client.get_available_maps()

    print(f"[현재 맵] {current_map}\n")
    print("[사용 가능한 맵 목록]")
    for i, m in enumerate(sorted(available_maps), 1):
        short = m.split("/")[-1]
        print(f"{i:2d}. {m}  (short: {short})")


if __name__ == "__main__":
    main()
```